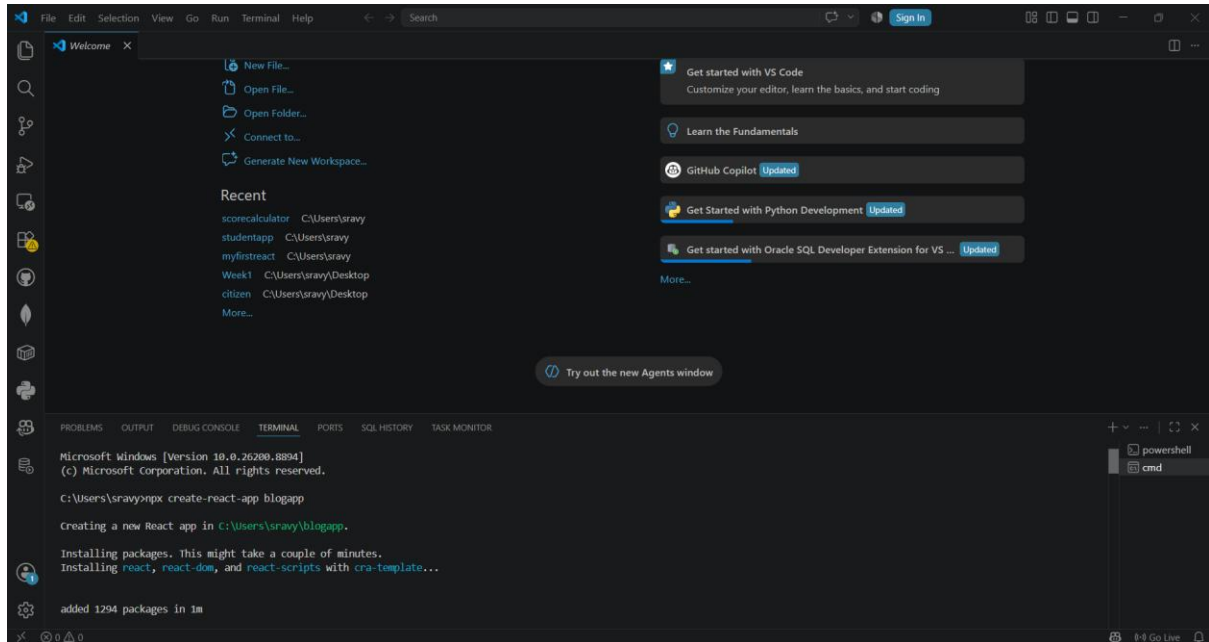


React

4. ReactJS-HOL

Step 1: Creating the React Application (blogapp) Using Create React App



The screenshot shows the Visual Studio Code interface with the terminal window open. The terminal displays the following commands and output:

```
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

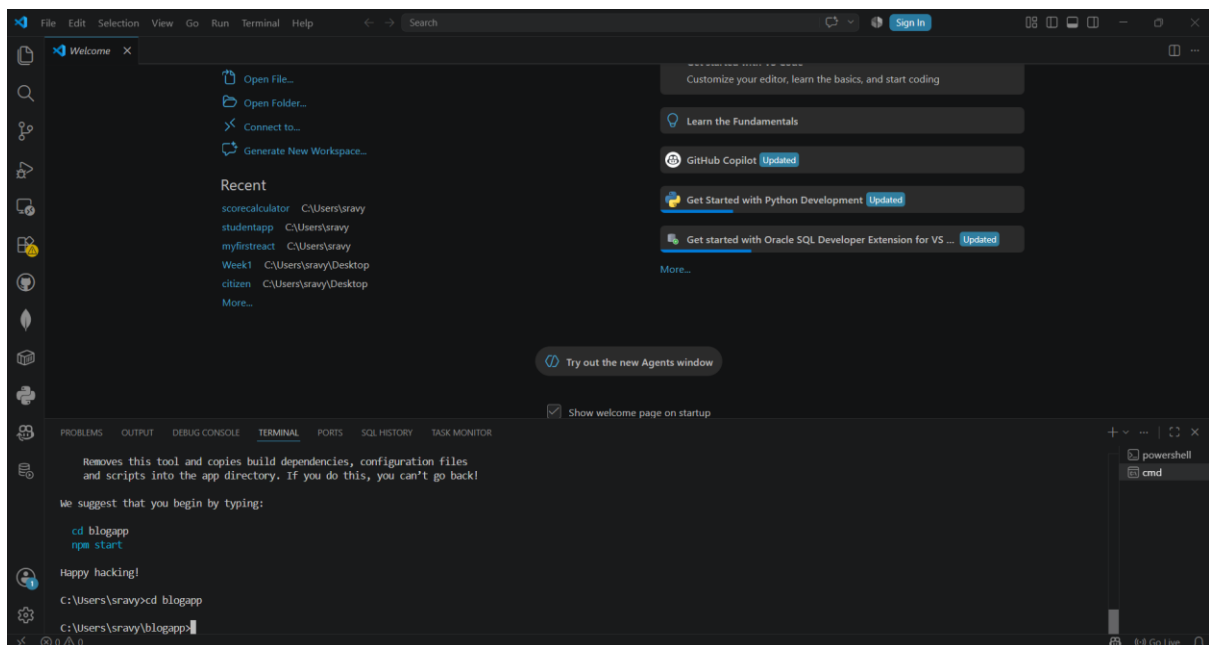
C:\Users\srahy>npm create-react-app blogapp

Creating a new React app in C:\Users\srahy\blogapp.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...

added 1294 packages in 1m
```

Step 2: Successfully Creating the React Project and Navigating to the Project Directory



The screenshot shows the Visual Studio Code interface with the terminal window open. The terminal displays the following commands and output:

```
Removes this tool and copies build dependencies, configuration files
and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

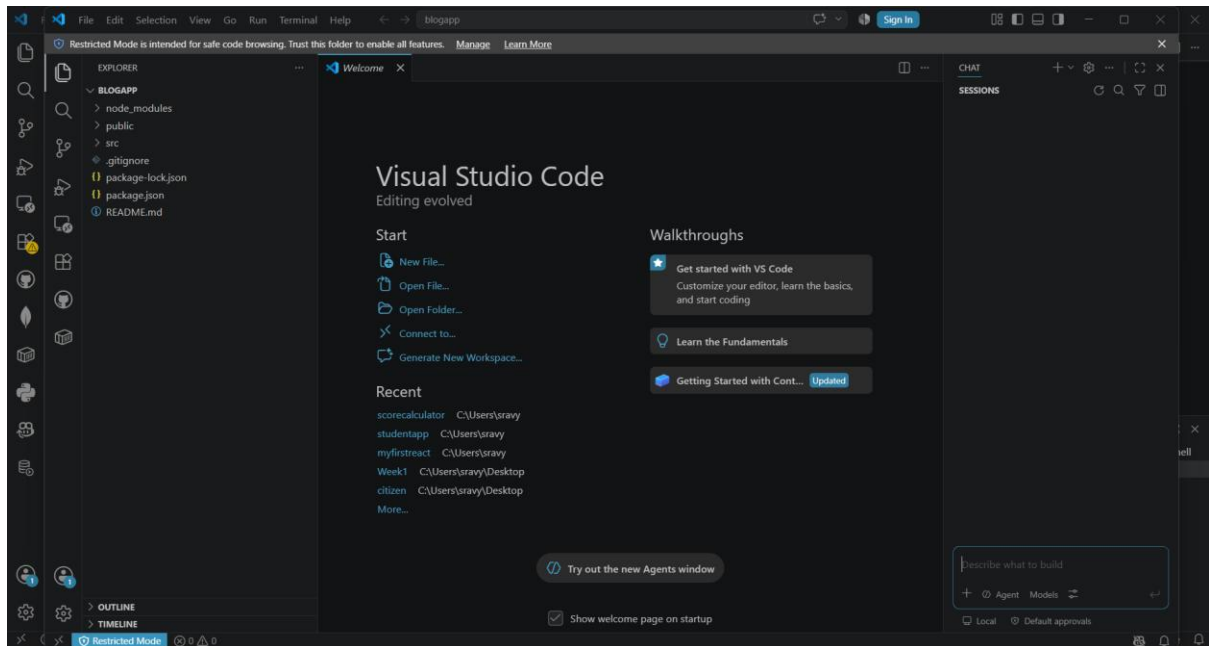
  cd blogapp
  npm start

Happy hacking!

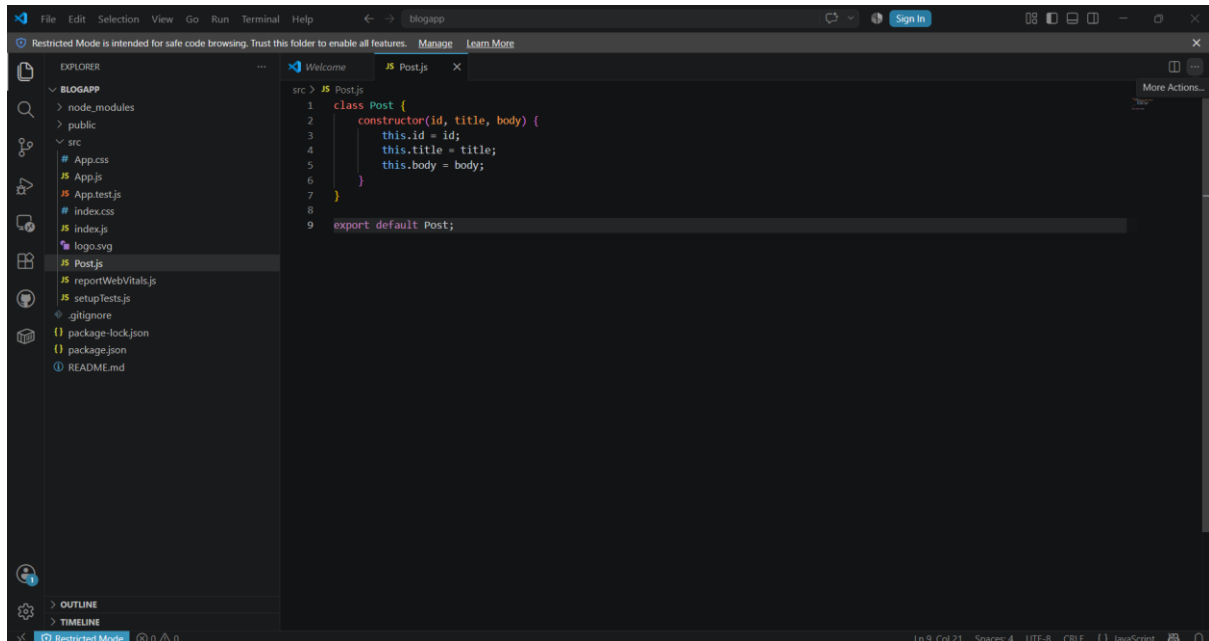
C:\Users\srahy>cd blogapp
C:\Users\srahy\blogapp>
```

COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 3: Opening the React Project (blogapp) in Visual Studio Code

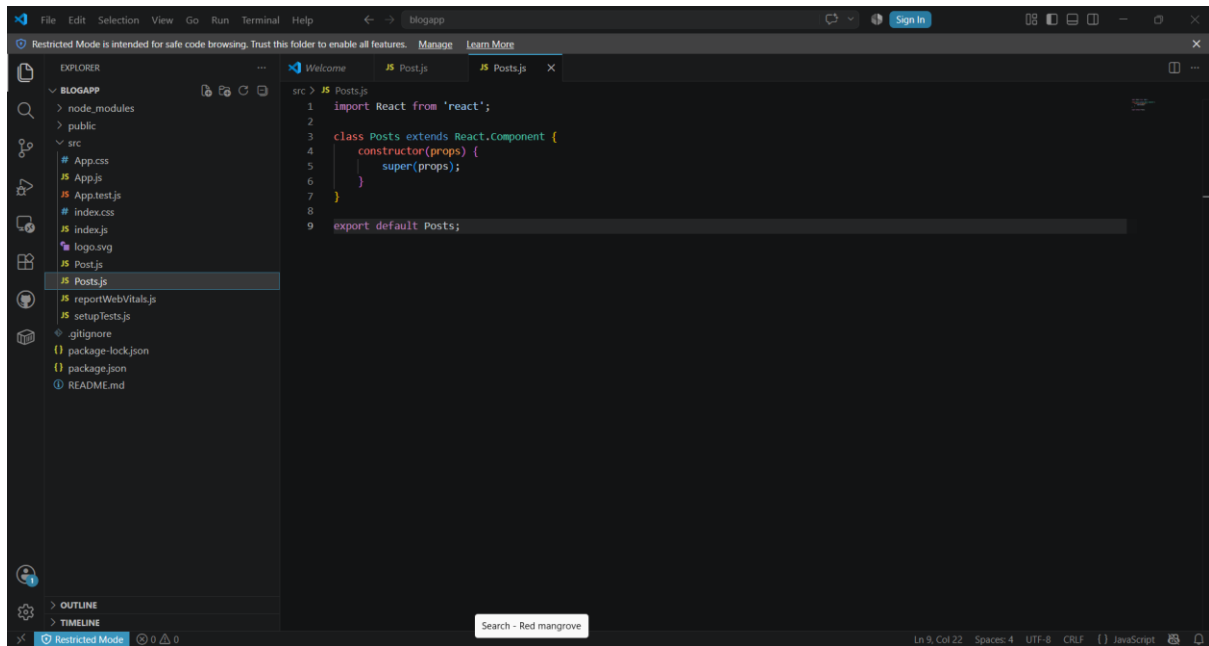


Step 4: Creating the Post.js Class Component



COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

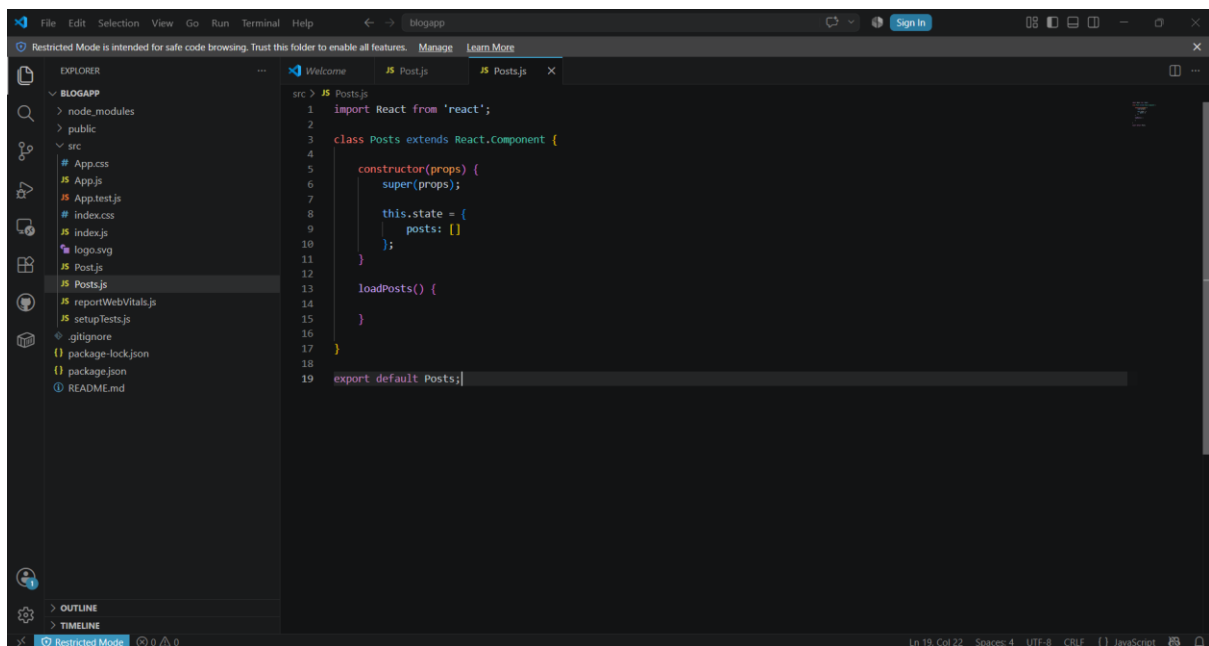
Step 5: Creating the Posts.js Component



The screenshot shows the Visual Studio Code editor with the 'BLOGAPP' project open. The Explorer sidebar on the left shows the file structure, with 'Post.js' selected under the 'src' folder. The main editor window displays the code for 'Post.js'.

```
src > JS Post.js
1 import React from 'react';
2
3 class Posts extends React.Component {
4   constructor(props) {
5     super(props);
6   }
7 }
8
9 export default Posts;
```

Step 6: Initializing the Component State and Creating the loadPosts() Method

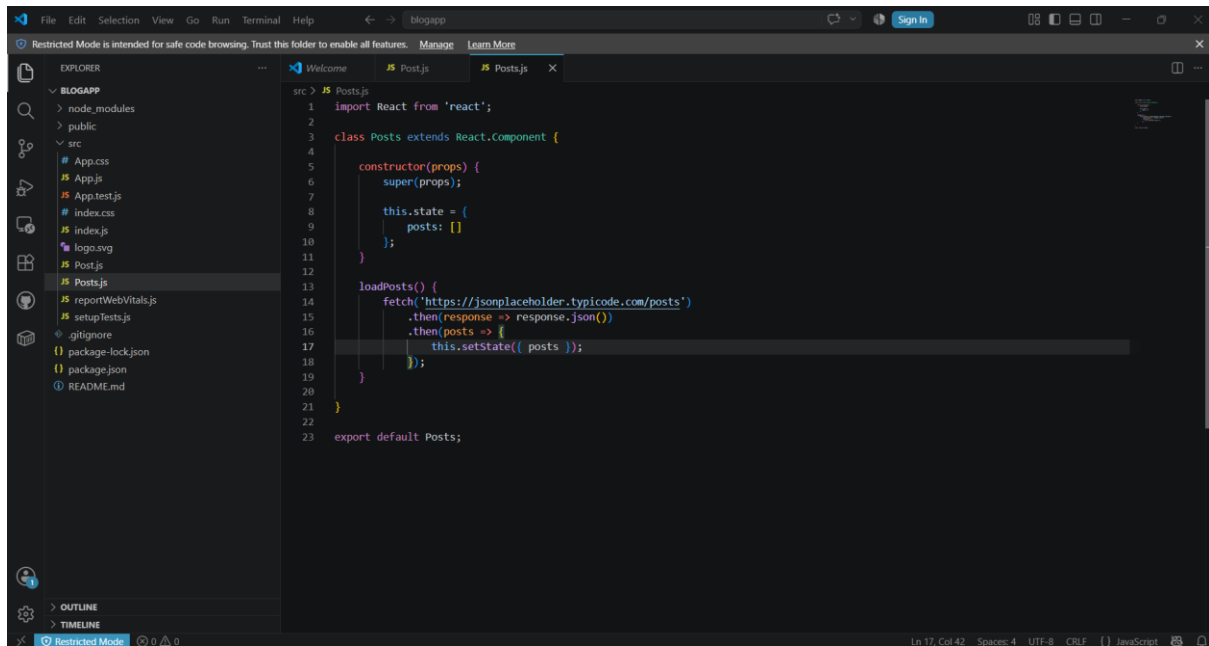


The screenshot shows the Visual Studio Code editor with the 'BLOGAPP' project open. The Explorer sidebar on the left shows the file structure, with 'Post.js' selected under the 'src' folder. The main editor window displays the code for 'Post.js'.

```
src > JS Post.js
1 import React from 'react';
2
3 class Posts extends React.Component {
4   constructor(props) {
5     super(props);
6
7     this.state = {
8       posts: []
9     };
10  }
11
12  loadPosts() {
13  }
14
15  }
16
17  export default Posts;
```

COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

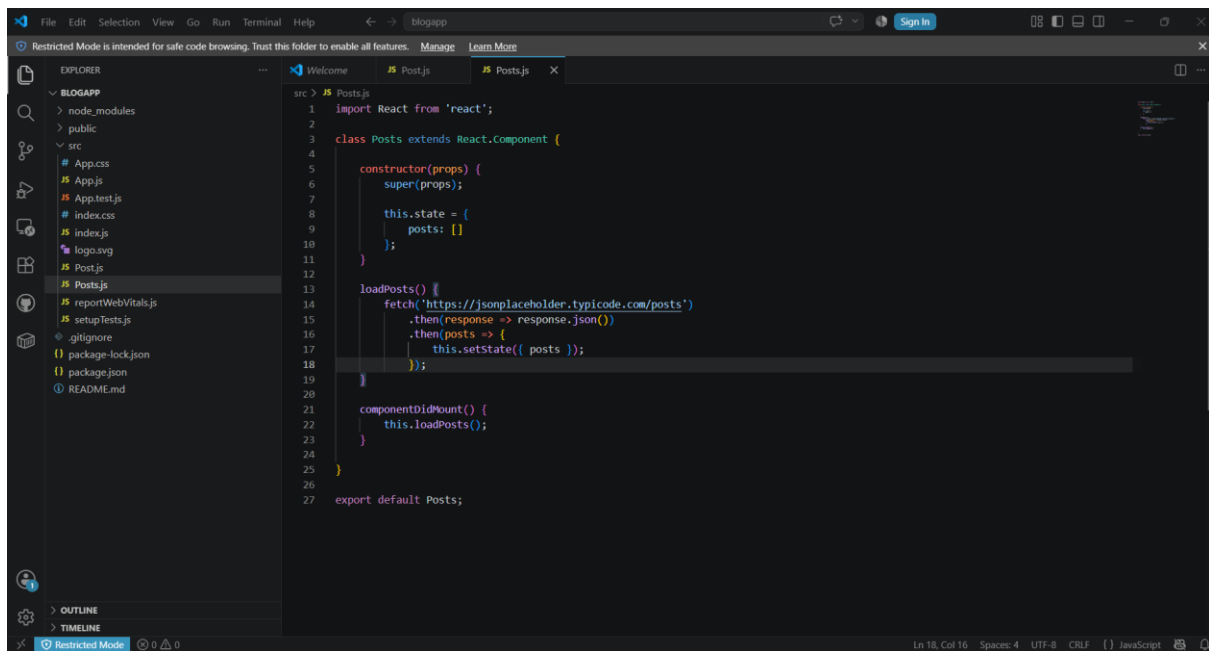
Step 7: Implementing the loadPosts() Method to Fetch Data from the JSONPlaceholder API



The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `Posts.js` file with the following code:

```
1 import React from 'react';
2
3 class Posts extends React.Component {
4
5   constructor(props) {
6     super(props);
7
8     this.state = {
9       posts: []
10    };
11  }
12
13  loadPosts() {
14    fetch('https://jsonplaceholder.typicode.com/posts')
15      .then(response => response.json())
16      .then(posts => {
17        this.setState({ posts });
18      });
19  }
20
21 }
22
23 export default Posts;
```

Step 8: Implementing the componentDidMount() Lifecycle Method

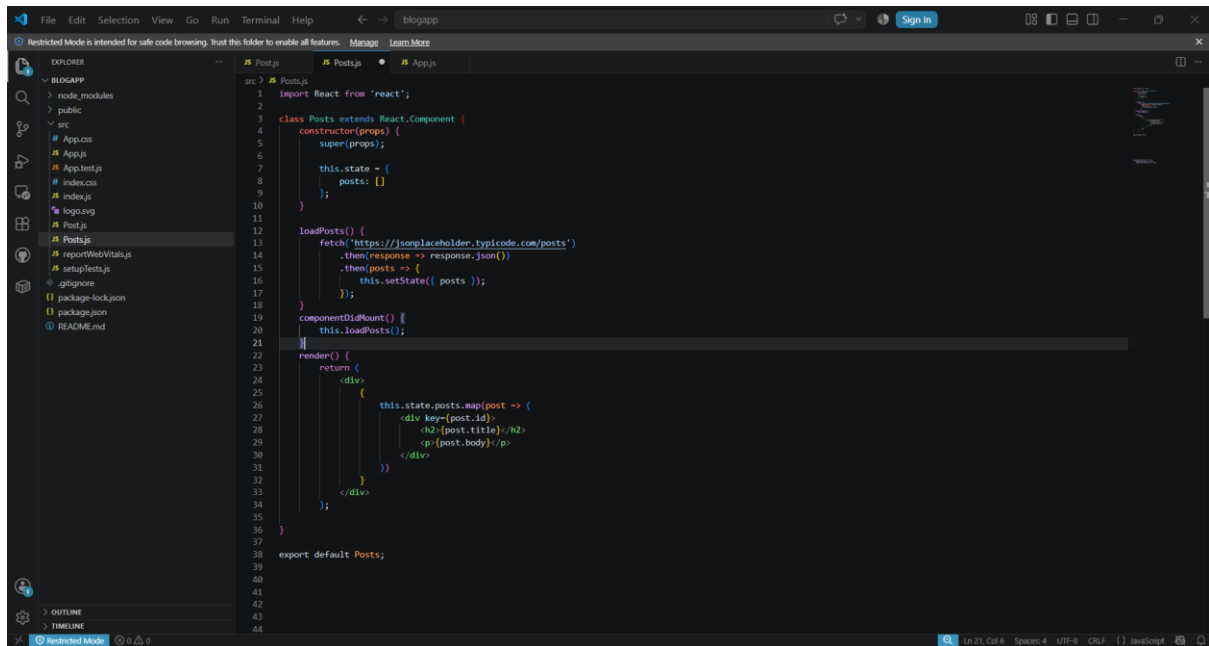


The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `Posts.js` file with the following code:

```
1 import React from 'react';
2
3 class Posts extends React.Component {
4
5   constructor(props) {
6     super(props);
7
8     this.state = {
9       posts: []
10    };
11  }
12
13  loadPosts() {
14    fetch('https://jsonplaceholder.typicode.com/posts')
15      .then(response => response.json())
16      .then(posts => {
17        this.setState({ posts });
18      });
19  }
20
21  componentDidMount() {
22    this.loadPosts();
23  }
24
25 }
26
27 export default Posts;
```

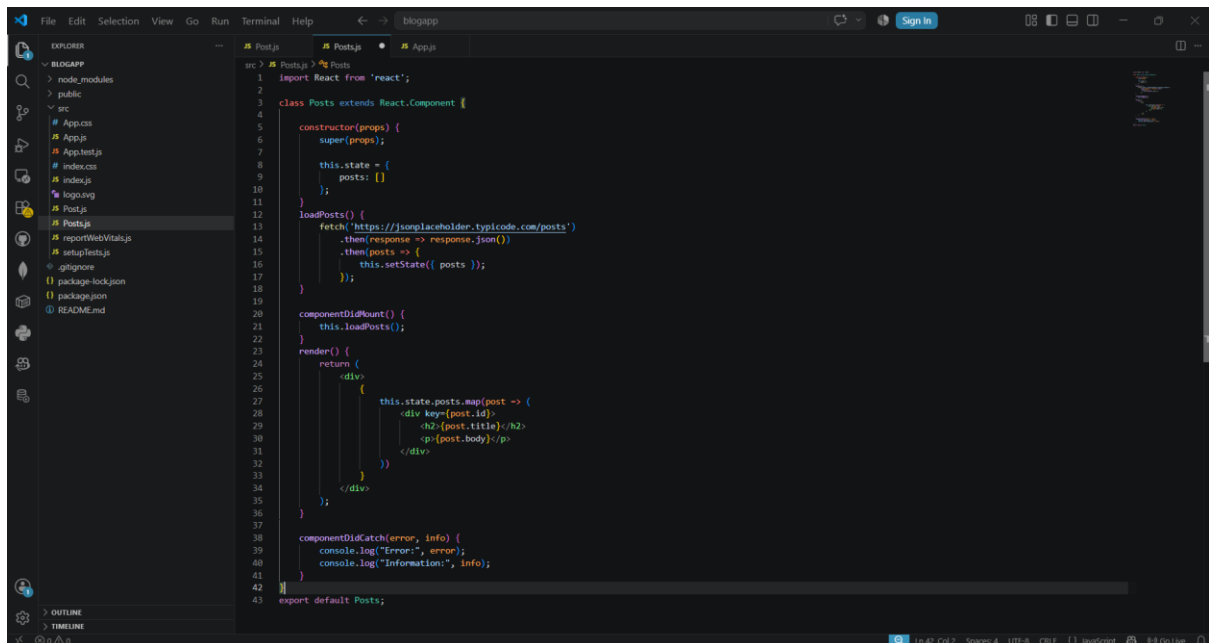
COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 9: Implementing the render() Method to Display Posts



```
1 import React from 'react';
2
3 class Posts extends React.Component {
4   constructor(props) {
5     super(props);
6
7     this.state = {
8       posts: []
9     };
10  }
11
12  loadPosts() {
13    fetch('https://jsonplaceholder.typicode.com/posts')
14      .then(response => response.json())
15      .then(posts => {
16        this.setState({ posts });
17      });
18  }
19
20  componentDidMount() {
21    this.loadPosts();
22  }
23
24  render() {
25    return (
26      <div>
27        {
28          this.state.posts.map(post => (
29            <div key={post.id}>
30              <h2>{post.title}</h2>
31              <p>{post.body}</p>
32            </div>
33          ))
34        }
35      </div>
36    );
37  }
38
39  export default Posts;
```

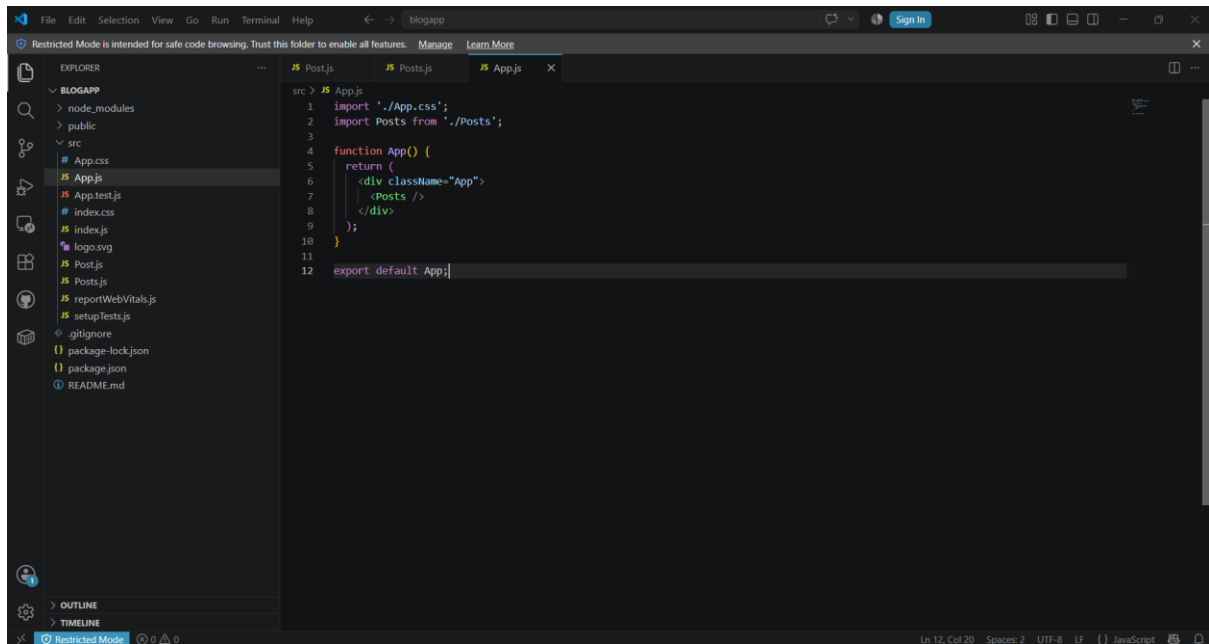
Step 10: Implementing the componentDidCatch() Lifecycle Method



```
1 import React from 'react';
2
3 class Posts extends React.Component {
4   constructor(props) {
5     super(props);
6
7     this.state = {
8       posts: []
9     };
10  }
11
12  loadPosts() {
13    fetch('https://jsonplaceholder.typicode.com/posts')
14      .then(response => response.json())
15      .then(posts => {
16        this.setState({ posts });
17      });
18  }
19
20  componentDidMount() {
21    this.loadPosts();
22  }
23
24  render() {
25    return (
26      <div>
27        {
28          this.state.posts.map(post => (
29            <div key={post.id}>
30              <h2>{post.title}</h2>
31              <p>{post.body}</p>
32            </div>
33          ))
34        }
35      </div>
36    );
37  }
38
39  componentDidCatch(error, info) {
40    console.log("Error:", error);
41    console.log("Information:", info);
42  }
43
44  export default Posts;
```

COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 11: Updating App.js to Render the Posts Component



```
1 import './App.css';
2 import Posts from './Posts';
3
4 function App() {
5   return (
6     <div className="App">
7       <Posts />
8     </div>
9   );
10 }
11
12 export default App;
```

Step 12: Running the React Application and Displaying the Posts in the Browser

